

Internet Protocols IV

Juliusz Chroboczek

20 October 2025

1 Introduction to security

The notion of *security* only makes sense within a certain context. For example, the web page with these lecture notes is not protected by a password; since these lecture notes are public, this does not constitute a security issue.

More precisely, the context of a security claim is defined by two sets of proprieties:

- the *security policy*, which described the desired properties (for example, the lecture notes are public but the exam subject is only available to the lecturer and the TAs);
- the *attack model*, which describes the moves the attacker is capable of (for example, I assume that students might try to guess the password of a password-protected exam subject, but that they won't physically break into my office).

Of course, a large part of the job of securing a protocol consists in ensuring that the security policy and the attack model are realistic. Many attacks are due not to cryptographic or software flaws, but to an overly permissive security policy or an overly restrictive attack model.

2 Security properties

There are a number of standard security properties that we aim to satisfy.

Confidentiality Confidentiality is probably the most intuitive security property: it means that the attacker is unable to learn the content of the communication.

It used to be considered that confidentiality without authenticity is useless, as an attacker can mount a *Man in the middle* (MITM) attack, as described during the lecture. This point of view is no longer universal, as confidentiality without authentication (“ad-hoc encryption”, or “better than nothing security”) is fairly successful at thwarting mass surveillance.

Forward secrecy Forward secrecy is a particular case of confidentiality: it is confidentiality where the attacker is future me. Imagine an attacker (for example a security agency) that records encrypted traffic; the attacker might later obtain the encryption key, either by breaking into a home or through the courts. Forward secrecy is the property that encrypted traffic cannot be decrypted in the future even if the encryption keys are disclosed to the attacker.

Authenticity Authenticity is the property that a message does come from the claimed sender. For example, the university will, someday, send a message to their bank asking to send me my salary; the bank will hopefully check the authenticity of the message in order to ensure that I (or anyone else) cannot fake the transfer order.

Integrity Integrity is the property that a message has not been modified between the sender and the receiver. Coming back to the previous example, the amount of money that the university orders to be transferred must remain unchanged between the sender (the university) and the receiver (the bank).

Availability Availability is the property that a service remains available even though the attacker is trying to make it fail. As we have seen during the lecture, there are a number of simple (but illegal¹) methods that can make a network service unavailable; since there's not much that can be done, the security community sometimes neglects this property.

An attack against availability is called a *denial of service* (DoS). We have seen during the lecture two specific cases of DoS, the *distributed denial of service* (DDoS) and the *reflection attack*.

3 Security primitives, constructions and protocols

Cryptographers provide us with *security primitives*, mathematical constructions that have some well-defined properties. Security primitives tend to be difficult to use, so cryptographers also provide us with *constructions*, built upon primitives, and easier to use. With these constructions, we build *security protocols*, which ensure that our network protocols satisfy the desired security properties (under a given attack model).

3.1 Primitives

Some examples of security primitives.

3.1.1 Block encryption

Given a key k , a *block encryption* is a bijective function E_k that maps a block of n bytes, the *cleartext*, to a block of n bytes, the *ciphertext*, where n is the *block size*. A block encryption function has the property that it is unfeasible to guess B given $E_k(B)$ without knowledge of k , even if the attacker has access to a large number of pairs $(B, E_k(B))$.

The current standard block encryption function is the *Advanced Encryption Standard* (AES), formerly known as *Rijndael*.

Block encryption is not directly usable by cryptographic protocols, for at least two reasons. First, block encryption operates on fixed-sized blocks, which is not flexible enough for most applications. Second, block encryption maps two identical pieces cleartext to the same ciphertext, which gives too much information to the attacker.

1. Please don't get in trouble.

3.1.2 Cryptographic hash

A *hashing function* is function from sequences of bytes to a finite space:

$$h : A^* \longrightarrow 2^n$$

that is *statistically injective*: given $x \neq y$, it is probable (but not certain) that $h(x) \neq h(y)$. A pair (x, y) such that $h(x) = h(y)$ is called a *collision*.

A *cryptographic hashing function* is a hashing function for which finding a collision is unfeasible, i.e. we may assume that the attacker is unable to find a collision. The currently recommended cryptographic hash function is SHA-2, other choices include SHA-3 and the *Blake* family of functions.

Many cryptographic functions (including SHA-2) are vulnerable to *extension attacks*: given $h(A)$, it is possible to compute $h(A \cdot B)$, where $A \cdot B$ is the concatenation of A and B , without knowing A . This implies that cryptographic hash functions cannot be used for integrity protection without using a suitable construction.

3.2 Constructions

As we have seen above, most primitives are not suitable for use in cryptographic protocols. Cryptographers provide us with higher-level *constructions*.

3.2.1 HMAC

A Message Authentication Code (MAC) is a function

$$\text{MAC}_k : A^* \longrightarrow 2^n$$

where k is a secret, that can be used to verify the authenticity and integrity of messages: it is impossible to compute $\text{MAC}_k(A)$ without knowing k , and if $\text{MAC}_k(A) = \text{MAC}_k(B)$, then it may be assumed that $A = B$.

A properly constructed MAC can be used for authenticity and integrity protection with a shared secret. Suppose that Alice and Bob have a shared secret k , and Alice wants to send Bob a message m . Alice computes the MAC of m , and sends to Bob the pair

$$(m, \text{MAC}_k(m)).$$

Bob receives a pair (m, h) , computes the MAC of the received message, and compares it with the value received. If $h = \text{MAC}_k(m)$, then the authenticity and integrity of the message can be assumed: the received message is the exact copy of a message produced by somebody with knowledge of the key k .

Constructing a correct MAC from a cryptographic hash function is not completely trivial. Suppose that Alice were to define

$$\text{BADMAC}_k(m) = h(k \cdot m)$$

where h is a cryptographic hash function vulnerable to extension attacks. Suppose now that Alice sends the pair

$$(m, \text{BADMAC}_k(m))$$

which is intercepted by Damian. Damian appends a post-scriptum:

$$m' = m \cdot p$$

Since h is vulnerable to extension attacks, Damian can compute $h(k \cdot m \cdot p) = \text{BADMAC}_k(m')$ without knowledge of k . Bob will verify the MAC, and assume that the message is authentic and integrity-protected.

HMAC is a MAC construction based on a cryptographic hash function that is invulnerable extension attacks. HMAC is defined in RFC 2104.

3.2.2 Encryption modes

As we have seen in Section 3.1.1, block encryption is not suitable for direct usage by cryptographic protocols, for at least two reasons. In order to provide useful constructions, cryptographers combine block encryption with *modes of operation* in order to provide encryption algorithms that work on streams of arbitrary length and avoid exposing repeated data within a stream (but not necessarily between streams).

There is a whole fauna of modes. Two modes are common in network protocols:

- *Counter mode* (CTR), which is simple to implement and can be easily parallelised;
- *Galois/Counter mode* (GCM), which is more complex but provides both encryption and integrity protection.

4 Protocols

Cryptographic constructions can be directly used in network protocols; in practice, however, it is often the case that a standard cryptographic protocol is available that solves a part of the security needs of the application. When that is the case, it is essential for the application designer to have a full understanding of the attack model and the security properties assumed by the cryptographic protocol (and its implementation).

In the first part of this course, we will use a standard security protocol, TLS. In the second part, we will implement our own security protocols based on standard constructions.

4.1 TLS and HTTPS

TLS (formerly SSL) is a standard protocol. TLS has the following properties:

- it can only protect streams of bytes, for example TCP streams (a variant called DTLS can protect client-server streams of datagrams);
- it is restricted to client-server communication;
- it provides confidentiality;
- forward secrecy is optional in TLS 1.2, and guaranteed in TLS 1.3;
- authentication is optional.

HTTP implemented over TLS is called HTTPS. When used in HTTPS, TLS has the following additional restriction:

- HTTPS authenticates the server, but not the client.

We will manipulate TLS in the next lab session, and explore the notion of *X.509 certificate*, which is a data structure that is used to authenticate the HTTPs server.

4.2 Username/password authentication

The profile of TLS used in HTTPS authenticates the server only; thus, there is a need for a mechanism to authenticate the client.

The most common such mechanism is *username/password authentication*. The server has access to a database of user accounts, where each row contains (among other data) a username and a password. When a client connects, it sends a username/password pair to the server, which verifies the password against the database.

Username/password authentication has a number of weaknesses. The most serious is that users tend to use the same password on multiple servers, which implies that a password leak has serious consequences. As the server is typically maintained by underpaid interns who have followed one semester of PHP programming, password leaks are a realistic concern.

Failure of other mechanisms There are a number of other mechanisms that have been proposed to replace username/password authentication, but they have failed to gain acceptance. With all their flaws, passwords are a familiar concept, that users have accepted, and in practice it is difficult to impose a more secure mechanism.

4.3 Token authentication

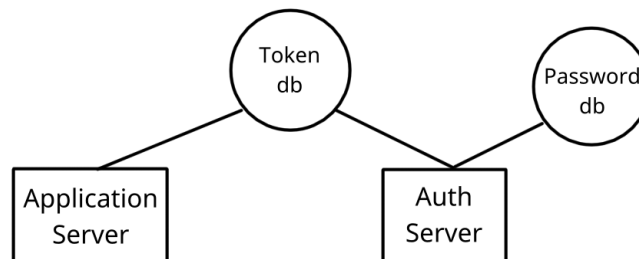


Figure 1. Token authentication

Token authentication is an improvement over username/password which aims to minimise the likelihood of password leaks. In token authentication (Figure 1), there are two distinct servers, the *authentication server* and the *application server*. The authentication server has access to the password database, and both the authentication server and the application server share a token database, initially empty.

When a user connects to the service, the client first sends a request to the authentication server, which consults the password database, and, if the user is authorised, generates a *token*, an opaque, unguessable string of bytes. It inserts a row in the token database, that contains the token, and any

other data required to use the service (such as the username, the user id, the user's permissions, etc.).

The client then connects to the application server, and authenticates using the token (the password is never sent to the application server). The application server consults the token database, and if the token is found, it grants access to the client.

The main advantage of this mechanism is that only the authentication server (written in a proper programming language by a security expert) has access to the password database; the application server (written in PHP or Python by a bunch of underpaid interns) only has access to the token database. If a vulnerability in the application server is exploited, only the token database is leaked, which can be revoked without serious consequences for the users. Token authentication is used by a number of currently fashionable protocols, notably *OAuth2* and *OpenID Connect*.

There exists a variant of this technique, using *cryptographic* or *stateless* tokens, which avoids the need for a token database, but makes it difficult to revoke tokens. We shall implement stateless tokens during this week's lab.